

Documentation-Driven Threat Analysis

Documentation Authoring Guide

Revision 7 - Rebuild R8

Nuova costruzione della guida DDTA per la scrittura della documentazione

Il documento viene ricostruito progressivamente da una baseline vuota. Ogni nuova sezione sarà aggiunta solo dopo revisione del significato metodologico e verifica dell'integrità delle pagine già approvate.

1 Step 0 - Project Problem Framing

RIQUADRO TEMPORANEO - DA CANCELLARE PRIMA DELLA VERSIONE FINALE

Guida di partenza usata come riferimento genealogico della riscrittura:

DDTA_DOCUMENTATION_AUTHORING_GUIDE_R6_CANDIDATE_R2.tex

Questo riferimento serve solo durante la ricostruzione progressiva. Il contenuto della nuova guida non viene importato automaticamente: ogni regola viene riesaminata mentre viene applicata al caso DermaTriage.

1.1 Che cos'è il framing

Il **Project Problem Framing** mette a fuoco il problema — o la classe di problemi — che il progetto deve affrontare e il contributo principale che intende portare entro un boundary governato. Non è un tipo documentale L1: resta una **precondizione dell'autoring DDTA** e non sostituisce MacroRequirement, Decision o FunctionalRequirement. Il boundary può contenere responsabilità che verranno rese esplicite solo nei MacroRequirement successivi. Stare dentro il boundary non significa che ogni responsabilità, attore, integrazione o fase del ciclo di vita debba comparire nella frase finale di framing. Il framing deve rendere riconoscibile il problema; non deve anticipare il catalogo delle responsabilità del progetto.

Domanda fondamentale

Quale problema o classe di problemi deve affrontare il progetto, entro quale boundary significativo, se rimuoviamo temporaneamente le scelte di soluzione correnti?

1.2 Regole di authoring

Quello che va scritto è il problema, non la pipeline o il modello scelto per risolverlo. Si può indicare il risultato atteso a livello di problema, ma senza trasformarlo in una sequenza di requisiti. Le regole operative, per esempio una soglia o un fallback, restano fuori: appartengono a livelli successivi. Ogni significato aggiunto deve essere supportato dalle fonti, e aggiungere dettaglio per riempire i campi non serve.

Regola di neutralizzazione della soluzione

Per riconoscere il problema si mettono *temporaneamente* da parte le scelte tecniche correnti. Questo non autorizza a perdere informazione: le scelte realmente governate verranno recuperate al semantic owner corretto nei livelli successivi.

1.3 Procedura umana di costruzione

1. **Leggere le fonti autorizzate prima di scrivere.** Identificare frasi che descrivono scopo, problema, outcome di dominio e boundary, senza assumere che ogni dettaglio sorgente appartenga al framing.
2. **Separare problema e soluzione.** Mettere temporaneamente da parte i dettagli di realizzazione, per esempio una pipeline o una soglia, come possibili contenuti di livello successivo.
3. **Formulare il problema in linguaggio di dominio.** La frase deve restare vera anche se le scelte tecniche correnti vengono sostituite, salvo che una specifica tecnologia costituisca essa stessa un vincolo governato del problema.
4. **Controllare il boundary senza trasformarlo nel framing.** Non ampliare il progetto con obiettivi, attori o responsabilit  che le fonti non autorizzano. Allo stesso tempo, non copiare automaticamente nel framing finale tutto cio' che appartiene al boundary: una responsabilit  puo' essere interna al progetto e restare correttamente rinviata al livello MacroRequirement.
5. **Fermarsi quando il problema e' riconoscibile.** MacroRequirement, Decision e FunctionalRequirement arrivano dopo.

Gate di accettazione del framing

Prima di accettare il testo verificare:

- se tutte le scelte tecniche correnti cambiassero, il problema descritto continuerebbe a esistere?
- il testo descrive un problema/contributo di progetto oppure sta gia' prescrivendo una soluzione?
- sono stati introdotti significati non supportati dalle fonti autorizzate?
- ci sono dettagli che possono essere rimandati senza perdere l'identita' del problema?

Se una risposta segnala contaminazione o informazione non autorizzata, occorre riformulare prima di procedere ai MacroRequirement.

Anti-pattern principali

- trasformare l'architettura corrente nella definizione del problema;
- elencare dettagli tecnici solo per rendere il framing piu' concreto;
- anticipare fallback o regole condizionali per spiegare casi speciali;
- copiare lo stesso significato che verra' poi riscritto integralmente nel primo MacroRequirement;
- introdurre obiettivi plausibili di dominio che le fonti non governano.

DermaTriage - framing ottenuto durante l'applicazione della guida

Il problema che DermaTriage deve affrontare e' il supporto al triage precoce di casi dermatologici relativi a lesioni potenzialmente oncologiche. Il progetto parte dalle informazioni gia' disponibili sul caso per determinarne l'urgenza e una priorita' operativa di presa in carico, con l'obiettivo di favorire un instradamento specialistico tempestivo.

Nell'esempio non compaiono dettagli di realizzazione come la pipeline a quattro stadi o il fallback senza immagine. Queste informazioni non vengono eliminate dal progetto: vengono rinviate al livello documentale che ne possiede il significato.

1.4 Profilo di esecuzione LLM

Il profilo LLM adatta la guida umana all'esecuzione automatica. Non e' una metodologia alternativa e non estende l'autorità DDTA. Il suo scopo e' contenere alcune tendenze ricorrenti dei modelli linguistici: completare oltre il necessario, anticipare la soluzione, riempire i campi per inerzia e spacciare una parafrasi per decomposizione.

1.4.1 Prompt canonico - Project Problem Framing

ROLE

You are helping with DDTA documentation authoring. The DDTA Documentation Authoring Guide is the methodological authority. Work only from the authorized project sources provided for this step. Do not import project truth from domain knowledge, earlier reconstructions, Base Analysis, threat models, tooling requirements, or implementation assumptions.

TASK

Derive the Project Problem Framing.

Framing is a precondition for authoring, not an L1 document type. Identify the problem or class of problems the project must address, together with the contribution it intends to make, while setting aside current solution choices.

SOURCE-FIRST RULE

1. Pull out statements supported by the sources that describe purpose, problem, domain outcome, actors, and meaningful boundary.
2. Separate problem meaning from current realization details.
3. Keep realization details, but mark them DEFERRED so they can be placed at the right semantic level later.

WHAT NOT TO PUT IN THE FINAL FRAMING

- names of components, models, algorithms, or protocols just because they exist;
- pipeline stages or architecture choices;
- thresholds, mappings, fallback rules, or interface details;
- responsibilities or goals the authorized sources do not support;
- details added only to make the framing look complete.

COUNTERFACTUAL CHECK

Ask: "If all current technical and architectural choices were replaced, would the stated project problem still exist?" If not, rework the framing unless the relevant choice is itself a governed problem boundary.

STOP RULE

Stop once the project problem and contribution are recognizable. Do not pre-write MacroRequirements, Decisions, or FunctionalRequirements.

OUTPUT

Return exactly the analysis schema and final-output schema defined in this section. Do not omit required fields. For REQUIRED-SLOT fields with no supported value, use "-".

Regola per l'LLM

Un'informazione presente nella source non appartiene automaticamente al framing. Il modello deve prima classificarne il semantic level: conoscere un dettaglio non e' una ragione sufficiente per anticiparlo.

Anche un significato correttamente incluso nel project boundary non appartiene automaticamente alla frase finale di framing. Se puo' essere rinviato ai MacroRequirement senza perdere l'identita' del problema, deve restare fuori dal framing finale.

1.5 Schema di output per l'LLM

Per il framing si tengono separati due livelli: la **worksheet di analisi** e l'**output documentale finale**. La worksheet serve a rendere verificabile il ragionamento del modello; non va copiata nella documentazione governata.

1.5.1 Classificazione dei campi

Campo	Classe	Regola
Authorized sources	REQUIRED-CONTENT	Elenco delle sole fonti autorizzate per il passo.
Source-supported problem meaning	REQUIRED-CONTENT	Fatti di problema direttamente sostenuti dalle fonti.
Deferred solution details	REQUIRED-SLOT	Dettagli riconosciuti ma rinviati; "-" se assenti.
Unsupported / ambiguous claims	REQUIRED-SLOT	Significati non autorizzati o ancora ambigui; "-" se assenti.
Meaningful boundary	REQUIRED-SLOT	Boundary governato; "-" se non stabilito separatamente.
Counterfactual result	REQUIRED-CONTENT	PASS o REWORK con motivazione breve.
Final framing	REQUIRED-CONTENT	Un blocco conciso di prosa governata.

1.5.2 Formato esatto richiesto

```
[DDTA FRAMING ANALYSIS]
AUTHORIZED SOURCES:
- ...

SOURCE-SUPPORTED PROBLEM MEANING:
- ...

MEANINGFUL BOUNDARY:
-

DEFERRED SOLUTION DETAILS:
- ...

UNSUPPORTED / AMBIGUOUS CLAIMS:
- ...

COUNTERFACTUAL CHECK:
PASS | REWORK
Rationale: ...

FRAMING VERDICT:
PROCEED | REWORK | STOP
Rationale: ...

[FINAL PROJECT PROBLEM FRAMING]
<one concise governed prose block>
```

1.6 Gate sull'output LLM

Controlli prima di accettare l'output LLM

- il blocco finale contiene solo significato già verificato nella worksheet;
- i dettagli marcati DEFERRED non ricompaiono nel framing finale senza una ragione esplicita;
- nessun campo vuoto viene riempito con una parafrasi di un altro campo;
- PROCEED è ammesso solo se il counterfactual check e la source-authority review sono superati;
- l'output LLM resta una proposta da sottoporre alla stessa review semantica prevista per l'autore umano.

Nota sui controlli quantitativi

Le metriche consolidate di similarità e retrieval del corpus (Jaccard su shingles, cosine TF-IDF, cosine su embeddings e BM25 per retrieval) verranno definite in una sezione metodologica dedicata. Non sostituiscono i gate semantici e non sono necessarie per accettare il primo framing di un corpus privo di altri elementi documentali.

Separazione tra diagnostica e decisione metodologica

Le metriche quantitative potranno segnalare sovrapposizioni o possibili rewording, ma non potranno decidere da sole se un elemento DDTA è corretto. La decisione resta affidata ai gate semantici previsti dalla guida.

2 Uso della metodologia: authoring nativo e reconstruction

DDTA nasce come metodologia per l'**authoring nativo della documentazione di progetto**. Il suo impiego naturale comincia insieme al progetto e ne accompagna la progressiva definizione: problema, responsabilit , decisioni, requisiti.

Ricostruire documentazione preesistente   invece un uso secondario, di validazione: serve a recuperare il significato governabile dalle fonti disponibili e a far emergere ambiguit , lacune e sovrapposizioni che una documentazione DDTA nativa avrebbe dovuto prevenire o quantomeno rendere esplicite.

Tenere separati i due percorsi

Le regole per leggere documentazione legacy non devono appesantire il percorso di authoring nativo.

Percorso	Punto di partenza e uso
NATIVE	Si parte dal problema governato e dalle responsabilit� che il progetto deve assumere. Le scelte di soluzione vengono introdotte ai livelli successivi quando diventano commitment governati.
RECONSTRUCTION	Si parte da fonti legacy autorizzate e si recupera soltanto il significato che esse supportano. Ambiguit�, conflitti e significati mancanti restano visibili invece di essere completati per inferenza.

Stessa semantica, diverso punto di ingresso

I costrutti DDTA non cambiano tra i due percorsi. Cambia il modo in cui si ottiene il significato da documentare: nel native authoring viene governato mentre il progetto prende forma; nella reconstruction viene recuperato da una documentazione che non era stata scritta secondo DDTA.

3 Step 1 - MacroRequirement

Un MacroRequirement copre una sola **responsabilita' macro durevole che il progetto deve possedere per affrontare il problema governato**. Non dice ancora come quella responsabilita' verra' realizzata, e non e' un contenitore per ogni capability importante della soluzione. Una capability non diventa MacroRequirement solo perche' e' stabile, distinguibile o tecnicamente sostituibile. Occorre verificare che il progetto debba possedere quel significato come responsabilita' macro, e non soltanto che la soluzione corrente abbia bisogno di un passaggio equivalente.

Domanda fondamentale

Quale responsabilita' macro deve assumersi il progetto per affrontare il problema governato? Che contributo stabile porta al problema o al boundary? Il progetto deve possedere proprio questa responsabilita', oppure essa esiste come capability distinguibile solo perche' la soluzione corrente ha scelto di decomporre il problema in questo modo? Quale parte del suo significato resta necessaria anche se cambia l'architettura con cui il problema viene affrontato?

Il framing non e' un catalogo di MacroRequirement. Un MR deve restare **riconducibile al problema e al boundary governato**, ma non serve che compaia parola per parola nella frase di framing. Se una nuova responsabilita' cambia davvero il problema o il boundary, allora si riapre il framing.

MR e scelte di soluzione

Una Decision puo' far nascere obblighi e capability reali senza per questo creare nuove responsabilita' macro. Perche' un elemento diventi MR, il suo significato deve essere governato e posseduto dal progetto al livello del problema o del boundary, indipendentemente dalla particolare decomposizione scelta per realizzarlo. La semplice possibilita' di sostituire il componente che oggi realizza una capability non e' sufficiente. Se una diversa architettura plausibile puo' affrontare lo stesso problema senza conservare quella capability come responsabilita' distinguibile, il candidato e' probabilmente una Decision, un FunctionalRequirement o un significato di livello successivo.

Confine con Decision

Una Decision restringe un solo MR fissando una posizione progettuale significativa: per esempio una policy, un responsibility boundary, una strategia o una scelta tecnologica/architetturale con conseguenze downstream. Se il candidato descrive gia' *come* una responsabilita' viene governata o realizzata attraverso una posizione di questo tipo, va valutato come Decision e non come nuovo MR.

Due errori opposti

Da una parte, non ogni sotto-problema introdotto dalla soluzione merita di diventare MR. Dall'altra, responsabilita' macro diverse non vanno compresse in un unico MR solo perche' oggi le realizza lo stesso componente o la stessa pipeline.

3.1 Procedura umana: dalla responsabilita' candidata al MR

Si parte dal framing e dal boundary: quali responsabilita' macro deve possedere il progetto per affrontare quel problema? La risposta non deve anticipare una soluzione. Nel native authoring i candidati emergono dal lavoro di definizione; nella reconstruction arrivano solo dalle fonti legacy autorizzate. In questa fase una lista anche sovrabbondante va bene.

Ogni candidato riceve un **Title** e un **Intent**. Se serve, si aggiungono Context o Scope, ma solo quando aiutano davvero a capire di quale responsabilita' si sta parlando. Prima di valutarne la forma, chiedere se il progetto deve possedere quel significato **come responsabilita' macro**, oppure se il candidato descrive una capability che esiste perche' una particolare Decision o decomposizione architetturale ha introdotto quel passaggio. Nel secondo caso il significato va preservato, ma valutato a un livello inferiore. In **RECONSTRUCTION** questa ownership non deve essere dichiarata letteralmente dalla fonte: deve risultare dal significato che la fonte assegna stabilmente al progetto, senza completarlo per inferenza. I candidati si confrontano poi tra loro: overlap, containment, possibili dipendenze e diversa profondita' di decomposizione contano piu' della descrizione isolata del singolo candidato. Prima di compilare il documento si classifica: KEEP, MERGE, SPLIT, LOWER LEVEL, REWORK oppure DEFER. Solo i KEEP passano alla stesura completa.

Forma di lavoro minima per un candidato

Candidate title:

...

Intent:

...

Context:

[se necessario per disambiguare]

...

Scope note:

[se necessario per separare ownership]

...

Possible dependency:

[se emerge una dipendenza non gerarchica]

...

Questa e' una worksheet di authoring, non un nuovo tipo documentale L1 e non sostituisce il MacroRequirement finale.

3.2 Test di separazione e classificazione

Prima di accettare un candidato come MR, verificarlo con un insieme di domande complementari. Nessuna singola domanda decide da sola il livello semantico.

Test	Domanda operativa
Contributo al problema	Che contributo macro porta al problema o al boundary?
Resilienza alla soluzione	Se venisse adottata un'architettura plausibile diversa per affrontare lo stesso problema, questa responsabilit� continuerebbe a esistere come responsabilit� macro distinguibile? Oppure esiste solo perche' l'architettura corrente ha introdotto quel particolare passaggio?
Singularita'	E' una responsabilit� sola o ne sta nascondendo due che potrebbero cambiare separatamente?
Valore distinto	Ha un significato proprio, oppure riscrive il framing o duplica un altro MR?
Dependency vs containment	Il significato governato di questa responsabilit� richiede il contratto semantico di un'altra responsabilit�, pur mantenendo ownership distinta? Oppure esiste soltanto un rapporto di containment, ordine operativo, dataflow o consumo di output?
Coerenza della futura famiglia	Le Decision che la restringerebbero appartengono allo stesso owner semantico? Il fatto che oggi non siano ancora definite non e' un problema.
Orizzonte di decomposizione	Le parti del candidato possono essere ristrette dallo stesso insieme di commitment governati, oppure una parte possiede gia' una Decision family supportata mentre un'altra deve fermarsi al livello MR? Una differenza persistente segnala un possibile hidden split; non giustifica il far procedere tutto il candidato insieme.
Stabilit� temporale	Regge oltre una singola release, configurazione o iterazione?

Come leggere il controfattuale

Non stiamo cercando una responsabilit  che esisterebbe in ogni soluzione immaginabile. Il controfattuale deve pero' essere abbastanza profondo da non limitarsi alla sostituzione locale del componente corrente. Chiedere se "classificare un'immagine" sopravvive al passaggio da EfficientNet a un altro classificatore non basta: entrambe le alternative possono appartenere alla stessa decomposizione architetturale. Occorre chiedere se, adottando una diversa architettura plausibile per affrontare lo stesso problema di progetto, quel significato rimarrebbe una responsabilit  macro che il progetto deve possedere. "Retraining EfficientNet-B4" resta chiaramente legato alla soluzione; "evoluzione controllata del comportamento del sistema" resta invece riconoscibile come responsabilit  anche se cambiano tecnologia e meccanismo di adattamento. Superare il test di resilienza e' necessario ma non sufficiente per diventare MR: il candidato deve superare anche il test di ownership macro e gli altri test di separazione. Nel percorso **RECONSTRUCTION** il controfattuale serve solo a verificare la dipendenza del significato dalla decomposizione corrente; non autorizza a progettare o assumere architetture alternative non presenti nelle fonti.

Dipendenza non e' contenimento

Un MR puo' dipendere semanticamente da un altro MR. La presenza di una dipendenza non giustifica il merge se le due responsabilit  mantengono ownership e significato distinti. Ordine di esecuzione, passaggio di dati, consumo di un output o collocazione nella stessa pipeline non sono da soli sufficienti per stabilire dependsOn. La relazione va dichiarata quando il significato governato del MR dipendente fa affidamento sul contratto semantico dell'altro MR. Un indizio forte di dipendenza esiste quando il significato o le condizioni di validita' della responsabilit  dipendente non possono essere espressi correttamente senza assumere il significato governato dall'altro MR. Se la fonte mostra soltanto che un output viene consumato successivamente, senza governare la relazione semantica tra le due responsabilit , dependsOn non va inferito automaticamente.

3.3 Campi del MacroRequirement

Una volta superata la separazione iniziale, il candidato viene scritto nella forma L2 completa. Ogni campo deve aggiungere significato governato; la presenza di uno slot nel template non autorizza a inventare contenuto.

Campo	Classe	Uso corretto
Title	REQUIRED-CONTENT	Nome leggibile della responsabilit� macro. Non un requisito operativo, non una soluzione.
Intent	REQUIRED-CONTENT	Dice quale responsabilit� macro assume il progetto e quale contributo porta.
Context	REQUIRED-CONTENT	Il minimo background per interpretare la responsabilit� senza ambiguit�. Non � una parafrasi dell'Intent.
Stakeholders	REQUIRED-SLOT	Ruoli di dominio che beneficiano della responsabilit�, la governano, la operano o ne sono materialmente coinvolti. "�" se non emerge un significato distinto.
Scope	REQUIRED-SLOT	Boundary IN/OUT, quando serve a prevenire overlap o responsibility leakage. "�" se non aggiunge nulla.
Assumptions / Constraints	REQUIRED-SLOT	Solo condizioni macro valide per l'intero branch. "�" se non ce ne sono.
dependsOn	REQUIRED-SLOT	Dipendenza semantica non gerarchica da un altro MR: il significato della responsabilit� fa affidamento sul contratto governato dell'altro MR. Ordine operativo, dataflow o semplice consumo di output non bastano. None se nessuna dipendenza semantica � governata.
Lifecycle / Authority	governance metadata	Stato e authority del documento, quando il corpus li usa. Non sostituiscono il significato del MR.

Quando un campo non va riempito

Se un campo non porta niente di nuovo, non va riempito riscrivendo Intent o Context con altre parole. Dove l'assenza   ammessa, " "   meglio di una frase di comodo.

Scope non   un elenco di cose escluse

Non serve elencare modelli, pipeline o soglie solo per dire che non appartengono all'MR. Le scelte di soluzione restano semplicemente ai livelli che le governano.

3.4 Template L2 e risultato della review

```
# <MR-ID> - <Title>
Lifecycle: <...>
Authority: <...> [se usato dalla governance del corpus]

## Intent
...
## Context
...
## Stakeholders
...
## Scope
...
## Assumptions / Constraints
...
## dependsOn
<MR-ID or None>
```

A compilazione fatta, split e merge vanno ricontrollati. Output diversi non bastano per tenere due MR separati. Condividere oggi un componente, dei dati o una sequenza operativa non basta per fonderli. Va controllato anche l'**orizzonte di decomposizione**. Se una parte del candidato dispone di commitment governati che permettono di procedere a Decision mentre un'altra parte resta semanticamente corretta solo fermandosi a MR, verificare esplicitamente se il candidato nasconde due responsabilit  distinte. Questa differenza   un segnale di review, non una regola automatica di split: puo' anche riflettere una fonte incompleta. Un commitment governato su una parte non deve trascinare automaticamente l'altra a un livello di decomposizione che la fonte non supporta.

DermaTriage - STOP AT MR

La documentazione supporta una responsabilit  macro di indicazione della destinazione specialistica, ma non governa in modo sufficiente il vocabolario delle destinazioni, la regola di selezione, gli eventuali input specifici, il fallback o la responsabilit  di booking/assignment. La presenza, nello stesso Adaptation Layer, di commitment governati relativi alla priorit  operativa non autorizza a usarli come Decision della responsabilit  specialistica. La risposta corretta   mantenere il significato specialistico disponibile e fermare quel branch: non si inventano Decision o FunctionalRequirement per completare la gerarchia.

Gate D1 - MR

PASS quando la responsabilit    stabile, distinta dagli altri MR, collocata al livello corretto e sufficientemente chiara da non ammettere letture materialmente diverse. Dopo PASS chiedere se, nello scope corrente, esistono ulteriori commitment governati che restringono specificamente questa responsabilit  semanticamente omogenea e consentono di derivarne una Decision senza importare significato da un'altra parte del candidato, da un MR vicino o dalla struttura della soluzione.

NO -> STOP AT MR. YES -> procedere alle Decision.

Un commitment che restringe soltanto una parte di un candidato aggregato non autorizza l'intero candidato a procedere. In quel caso occorre riesaminare prima lo split/merge.

3.5 Sufficienza semantica e diagnosi

Il gate non misura la lunghezza del testo. Chiede se, con il significato governato a disposizione, restano in piedi letture materialmente diverse della responsabilit , del suo boundary o del suo owner. In reconstruction conviene classificare le proposition critiche come:

AFFIRMED
DENIED
NOT SPECIFIED
CONFLICTING

NOT SPECIFIED non   FALSE, e CONFLICTING non autorizza a scegliere la lettura piu' comoda.

Diagnosi	Cosa segnala
Source / documentation gap	Il significato manca, � ambiguo o conflittuale gia' nella fonte.
Authoring-guide problem	Il significato si puo' rappresentare, ma domande e gate non guidano l'autore in modo stabile.
Metamodel pressure	Il significato � chiaro e governato; sono i costrutti DDTA a non riuscire a rappresentarlo.
Representation / L2 issue	L1 basta, ma la forma documentale non � chiara o pratica.
Downstream / BA issue	La documentazione regge, ma una rappresentazione analitica a valle perde significato.
No gap	Differenza presentazionale, oppure dettaglio fuori dallo scope governato.

Il progetto non si corregge per far funzionare la metodologia

Un dubbio emerso in review non autorizza a cambiare documentazione, guida o metamodel. Prima va diagnosticato dove sta il problema.

3.6 Profilo di esecuzione LLM

Il profilo LLM segue gli stessi criteri dell'autore umano. Rende solo espliciti i controlli che servono a evitare tre errori ricorrenti: promuovere a MR un dettaglio di soluzione, chiamare decomposizione una parafrasi, riempire tutti i campi anche dove il significato non e' governato.

3.6.1 Prompt canonico - MacroRequirement

ROLE

You are helping with DDTA documentation authoring. The DDTA Documentation Authoring Guide is the methodological authority. Work only from the project meaning authorized for this authoring step.

AUTHORING MODE

The caller must state one mode: NATIVE or RECONSTRUCTION.

NATIVE: start from the governed project problem, boundary and current project meaning. Implementation state does not become project authority just because it exists.

RECONSTRUCTION: recover project meaning only from the authorized legacy sources. Missing meaning is not repaired with domain knowledge, earlier DDTA reconstructions, Base Analysis, threat models, tooling needs or implementation assumptions.

TASK

Identify, separate, and author MacroRequirements. A MacroRequirement owns one durable macro responsibility the project must possess to address the governed problem within its boundary. It is not a container for every stable capability or important project concern.

CANDIDATE DISCOVERY

1. Identify candidate responsibilities without assuming that every stable capability is a MacroRequirement.
2. Give each candidate at least a Title and Intent before judging it.
3. Add Context, Scope, or a possible dependency only when they are needed to understand ownership or separation.
4. Ask whether the project must own that meaning as a macro responsibility, or whether it exists because the current solution decomposes another responsibility into that capability.
5. Preserve solution-induced obligations, but classify them LOWER_LEVEL when a technical or architectural commitment introduced that decomposition.

SEPARATION TESTS

For every candidate ask:

- What distinct macro contribution does it make to the governed problem or boundary?
- MACRO OWNERSHIP: must the project own this meaning as a macro responsibility, or does it exist because the current solution decomposes another responsibility into this capability? In RECONSTRUCTION, infer this only from source-supported project meaning; do not require literal ownership wording.
- SOLUTION RESILIENCE: if a plausibly different architecture were used to address the same project problem, would this meaning still exist as a distinguishable macro responsibility? Do not test only replacement of the current component by an equivalent component. In RECONSTRUCTION, use this only as a classification counterfactual; do not design or assume unsupported alternative architectures.
- Is it one responsibility, or several responsibilities that could change apart?
- Does it add its own meaning, rather than restating the framing or another MR?
- Is its relation to another MR a semantic dependency, containment, operational sequence, dataflow, or simple output consumption?
- Would the Decisions restricting it belong to one coherent semantic owner?
- DECOMPOSITION HORIZON: can the whole candidate be restricted by the same governed commitment family, or can one part proceed to Decisions while another must STOP AT MR? Treat a mismatch as SPLIT-PRESSURE, not an automatic split.
- Does the responsibility remain stable beyond one release or configuration?

No single PASS makes a candidate a MacroRequirement. Solution resilience or a coherent Decision family is not sufficient without macro ownership.

Do NOT require complete independence from other MR. A valid MR may depend on another MR through dependsOn while retaining separate ownership. Do not infer dependsOn from execution order, dataflow, or output consumption alone.

CLASSIFY BEFORE FULL AUTHORIZING

KEEP | MERGE | SPLIT | LOWER_LEVEL | REWORK | DEFER

Only KEEP candidates may become final MacroRequirements.

Regola per l'LLM

Conoscere un dettaglio della soluzione non gli assegna automaticamente un posto nel livello MR. Il modello deve prima stabilire quale responsabilit  governa quel significato, se il progetto deve possederlo come responsabilit  macro e se regge a un cambio plausibile della decomposizione architetturale. Un singolo test superato non basta per promuovere il candidato.

FULL MR AUTHORING

For each KEEP candidate produce:

- Title: REQUIRED-CONTENT
- Intent: REQUIRED-CONTENT
- Context: REQUIRED-CONTENT
- Stakeholders: REQUIRED-SLOT; use "-" if no distinct governed meaning exists
- Scope: REQUIRED-SLOT; use "-" if it adds no distinct boundary information
- Assumptions / Constraints: REQUIRED-SLOT; use "-" if none are governed
- dependsOn: REQUIRED-SLOT; use "None" when no semantic dependency is governed

FIELD NOVELTY RULE

Do not fill a field by paraphrasing another field. Scope is not a list of technical choices to exclude. Assumptions/Constraints must apply to the whole MR branch.

dependsOn is semantic and non-hierarchical. Do not infer it from narrative order, pipeline order, dataflow, shared data, or simple consumption of another MR's output. Use dependsOn only when the governed meaning of one MR relies on the semantic contract owned by another MR. A strong indicator exists when the dependent responsibility cannot be expressed correctly without assuming that governed meaning.

SEMANTIC SUFFICIENCY

Ask whether materially different interpretations of the responsibility can still survive. If yes, REWORK or STOP rather than inventing lower-level meaning. A stable MR may PASS even when lower-level Decisions or FR are NOT SPECIFIED.

BRANCH STOP RULE

After an MR passes, proceed to Decisions only when additional governed commitments specifically restrict that semantic responsibility.

A commitment that restricts another MR, or only one separable part of an aggregated candidate, does not authorize this branch to proceed.

If different parts of the candidate have different decomposition horizons, re-open SPLIT/MERGE before deciding PROCEED TO DECISIONS. A mismatch is a review signal, not an automatic split.

Otherwise STOP AT MR.

3.7 Worksheet richiesta all'LLM

La worksheet rende verificabile la classificazione dei candidati e resta separata dalla documentazione governata finale.

3.7.1 Formato esatto richiesto

```
[DDTA MR ANALYSIS]
AUTHORING MODE: NATIVE | RECONSTRUCTION
AUTHORIZED PROJECT BASIS:
- ...

CANDIDATE <n>
TITLE: ...
INTENT: ...
SOURCE / GOVERNED SUPPORT: ...
CONTEXT NOTE: ... | -
SCOPE NOTE: ... | -
POSSIBLE DEPENDENCY: ... | None
MACRO-OWNERSHIP: PASS | LOWER_LEVEL | UNCERTAIN
SOLUTION-RESILIENCE: PASS | REWORK
DISTINCT-RESPONSIBILITY: PASS | MERGE | SPLIT | REWORK
DECOMPOSITION-HORIZON: COHERENT | SPLIT-PRESSURE | NOT-YET-TESTABLE
SEMANTIC-LEVEL: MR | LOWER_LEVEL | UNCERTAIN
CLASSIFICATION: KEEP | MERGE | SPLIT | LOWER_LEVEL | REWORK | DEFER
RATIONALE: ...

[FINAL MACROREQUIREMENTS]
MR <id or candidate-id> - <Title>
Intent: ...
Context: ...
Stakeholders: ... | -
Scope: ... | -
Assumptions / Constraints: ... | -
dependsOn: <MR-ID> | None
MR VERDICT: PASS | REWORK
NEXT: PROCEED TO DECISIONS | STOP AT MR
```

Controlli prima di accettare l'output LLM

Prima di accettare l'output:

- ogni MR finale viene da un candidato classificato KEEP;
- ogni KEEP ha superato il macro-ownership test; solution resilience da sola non basta;
- nessun dettaglio marcato LOWER_LEVEL rientra come responsabilit  macro senza una nuova giustificazione;
- il solution-resilience test considera un cambio plausibile di architettura, senza progettare alternative non autorizzate in reconstruction;
- split e merge sono motivati sul significato, non sulla struttura attuale del software;
- una differenza di decomposition horizon viene riesaminata come possibile split, mai trattata come split automatico;
- dependsOn non viene dedotto dal solo ordine operativo, dataflow o consumo di output;
- Context e Scope portano informazione nuova invece di ripetere l'Intent;
- STOP AT MR si applica localmente alla responsabilit : commitment presenti in un'altra parte del candidato non autorizzano il branch a procedere;
- l'output LLM passa comunque dalla stessa review semantica dell'autore umano.

4 Step 2 - Decision

4.1 Che cos'è una Decision

Una **Decision** è un commitment progettuale significativo che restringe esattamente un MacroRequirement, attraverso una posizione, una policy, un boundary, una convenzione, una strategia, un placement o una scelta architettuale governata. Non sostituisce la responsabilità macro dell'MR e non anticipa gli obblighi operativi che apparterranno ai FunctionalRequirement.

Confine MR - Decision

MR: quale responsabilità macro possiede il progetto?

Decision: quale scelta governata restringe quella responsabilità senza cambiarne l'identità?

4.2 Discovery e promotion sono due passaggi diversi

Prima si cercano i significati che potrebbero essere Decision; solo dopo li si sottopone ai test di promozione. Un boundary interessante, una tecnologia importante o un comportamento ripetuto non diventano Decision solo perché sono stati scoperti.

Domande per la discovery

- La fonte contiene una policy, una convenzione, una strategia o una scelta architettuale?
- Esiste una separazione di fasi, un'integrazione, un lifecycle o un placement con significato proprio?
- Una condizione è accompagnata da una risposta progettuale selezionata?
- Un dettaglio tecnico nasconde un significato più astratto che resta comprensibile anche senza il nome concreto?

4.3 Worksheet minima della candidate

Candidate ID: ...
Parent MR: ...
Source evidence: ...
Minimum source-supported meaning: ...
Possible Decision meaning: ...
Possible lower-level owner: ...
Open source gaps: ...

Source-first

La candidate parte dal significato minimo realmente sostenuto dalla documentazione autorizzata. Il metodo distingue sempre ciò che la fonte stabilisce, ciò che non stabilisce e ciò che resta ambiguo.

DermaTriage - fallback senza immagine

La documentazione descrive una condizione senza immagine e un percorso basato sui sintomi. La candidate non nasce dalla parola “fallback”, ma dalla scelta progettuale più astratta: continuare la valutazione di triage anche quando l’immagine non è disponibile.

Domande da annotare, non da risolvere per intuizione

Quali output sono garantiti nel percorso senza immagine? Quali informazioni sintomatologiche sono necessarie? La fonte richiede equivalenza con il percorso image-based? Se la documentazione non risponde, il dubbio resta visibile.

4.4 Tre test prima della governance

4.4.1 MR-stability

Chiedere se il significato può cambiare mantenendo stabile la responsabilità macro dell'MR. Il confronto va fatto con l'identità semantica dell'MR, non con il vocabolario o la realizzazione corrente.

4.4.2 Neutralization

Rimuovere temporaneamente nomi di prodotti, modelli, endpoint, file, protocolli, parametri, soglie e binding concreti. Se non resta alcun significato progettuale, la candidate è normalmente lower-level.

Neutralization non è promotion

Se dopo la neutralizzazione resta una strategia o un boundary comprensibile, significa solo che esiste una candidate significativa. Non basta ancora a dimostrare che il progetto la governi come Decision autonoma.

4.4.3 Semantic-level test

Confrontare la candidate con MR identity, FunctionalRequirement, realization, interface detail, configuration, parameter/binding, data/model fact, verification evidence e source gap.

DermaTriage - retrieval storico

ChromaDB + embedding model + cosine + top-5 può neutralizzarsi in “historical-case retrieval”. La candidate sopravvive; resta però da capire se quel significato possiede governance autonoma o resta comportamento interno della pipeline.

4.5 Independent-governance test

La domanda centrale è: *quale evidenza nella documentazione mostra che il progetto possiede questa scelta proprio a questo livello semantico, invece di documentare soltanto la realizzazione corrente?*

Evidenza favorevole può venire da framing architetturale o di policy, rationale, alternative deliberate, boundary autonomi, conseguenze proprie, stabilità cross-document e significato di change/review. La ripetizione della stessa realizzazione in più punti non basta da sola.

4.6 Removal / replacement significance

Chiedere: se il significato neutralizzato venisse rimosso o sostituito lasciando stabile l'MR, la documentazione supporterebbe la conclusione che è cambiato un commitment governato, oppure soltanto la sua realizzazione?

Esempi di contrasto

Cambiare il nome di un endpoint non equivale a rimuovere una strategia di accesso. Cambiare un modello concreto non equivale a rimuovere uno stadio architetturale governato. Cambiare un parametro non equivale a cambiare una policy.

Domanda utile nella review

Se la fonte documenta due percorsi di accesso, stabilisce anche che entrambi debbano essere preservati come scelta di progetto, oppure descrive soltanto l'interfaccia corrente?

4.7 Decomposition e semantic owner

Una candidate non deve assorbire significati che possono cambiare o essere valutati indipendentemente. Valutare lo split quando una sub-parte:

- può variare indipendentemente;
- ha un semantic owner diverso;
- può essere revisionata o verificata separatamente;
- può cambiare senza cambiare il significato parent.

Questi segnali aiutano a individuare significati distinti, ma **non impongono da soli un branch Decision separato**. La struttura documentale deve restare utile anche dopo la derivazione dei requisiti.

Semantic owner non equivale automaticamente a branch autonomo

Una differenza concettuale può essere reale senza meritare una Decision autonoma. Dopo l'autoring degli FR, ogni branch deve dimostrare una conseguenza downstream distinguibile per implementazione, verifica, modifica indipendente o analisi. Se il significato vive soltanto come parte del contratto operativo di un branch sibling, riesaminare MERGE o REWORK; non creare un FR artificiale per giustificare lo split.

DermaTriage - P-scale

Il dominio P1-P4 e la regola urgency-to-priority sono significati distinguibili: il primo definisce il codominio della priorità, la seconda definisce come scegliere il livello. Questo non prova però che richiedano due branch Decision. Se il dominio non produce alcuna conseguenza downstream autonoma e diventa operativo soltanto nel branch che governa la derivazione, il significato va preservato in quel contratto senza mantenere una diramazione documentale separata.

4.8 Campi della Decision

Parent MR: <exactly one MR>

Context: <situazione progettuale o di dominio necessaria a capire perché la scelta
↔ e' rilevante>

Decision: <the governed commitment>

Consequences: <what follows from adopting the choice>

Regola per il Context

Il Context descrive il progetto, il dominio o la condizione in cui la scelta si applica. Non descrive il processo di analisi, non cita la documentazione come soggetto e non giustifica perché l'analista ha classificato l'elemento come Decision.

4.9 Esempio completo

DermaTriage - continuità del triage senza immagine

Parent MR: MR-01.

Context: un caso dermatologico può avere un'immagine della lesione, oppure no.

Decision: DermaTriage valuta il triage anche se manca l'immagine della lesione. In quel caso usa le informazioni sui sintomi disponibili sul caso.

Consequences: se l'immagine manca, il triage si può comunque fare. Il percorso basato sui sintomi può usare informazioni diverse; non deve per forza dare gli stessi risultati del percorso con immagine.

4.10 MR-prose Decision leakage

Durante la review, cercare strutture del tipo `condizione + risposta progettuale selezionata` dentro Context o Scope dell'MR. Se la risposta può cambiare mentre la responsabilità macro resta stabile, il testo MR può contenere una Decision che va spostata al proprio semantic owner.

Domanda di controllo

La frase nel Context descrive davvero il contesto della responsabilità, oppure prescrive già come il progetto risponde a una condizione particolare?

4.11 Decision-family regression

Dopo la review individuale, controllare la famiglia nel suo insieme:

- pairwise non-overlap e frammentazione inutile;
- candidate troppo ampie o troppo tecniche;
- significati governati rimasti senza owner;
- boundary di fase, integrazione, lifecycle e data relationship;
- MR-prose leakage;
- completezza rispetto alla source corrente.

Questa prima regression può chiudere la discovery delle Decision, ma la **struttura dei branch resta provvisoria fino alla derivazione degli FR**.

Downstream Utility Test - regression obbligatoria post-FR

Per ogni Decision che si vuole mantenere come branch autonomo, dopo la derivazione degli FR chiedere:

Se elimino questo branch come unità autonoma e trasferisco il suo significato nel branch che possiede i comportamenti che lo utilizzano, perdo una distinzione necessaria per implementazione, verifica, modifica indipendente o analisi?

YES: il branch possiede utilità downstream distinta. **NO:** la differenza è soltanto concettuale/documentale; riaprire la struttura e valutare MERGE o REWORK. Una Decision semanticamente vera non è sufficiente, da sola, a giustificare un branch autonomo.

Una candidate può quindi essere mantenuta, fusa, separata, riclassificata a un livello inferiore, rielaborata o lasciata in HOLD.

4.12 Closure layer-specific

SOURCE INSPECTION: COMPLETE | INCOMPLETE
 CANDIDATE DISCOVERY: EXHAUSTED FOR CURRENT SOURCE SET | OPEN
 DECISION FAMILY: PROVISIONAL | RESOLVED | HOLD
 LOWER-LEVEL SEMANTICS: OPEN | RESOLVED
 FR AUTHORING: NOT STARTED | STARTED | COMPLETE
 POST-FR DOWNSTREAM UTILITY: NOT RUN | PASS | REWORK

Domanda di chiusura

La family è RESOLVED solo quando nessun dubbio cambia identità o numero delle Decision e, se gli FR sono completi, il Downstream Utility Test non lascia branch autonomi privi di utilità distinta.

4.13 Profilo di esecuzione LLM

Il modello applica la stessa procedura dell'autore umano. Non crea authority, non completa source gap per plausibilit  e non trasforma una candidate in Decision perche' "sembra architetturale".

4.13.1 Prompt canonico - Decision

ROLE

You are helping with DDTA documentation authoring. Work on exactly one accepted MacroRequirement at a time. The authorized project basis is the only authority for project facts and commitments. The LLM is assistive and cannot create missing project meaning.

AUTHORING MODE

The caller must state NATIVE or RECONSTRUCTION.

In RECONSTRUCTION, recover meaning only from authorized legacy sources. Do not repair gaps using domain knowledge, previous DDTA answers, Base Analysis, threat models, tooling needs, or plausible design.

TASK

Discover, classify, and author the Decision family that restricts the supplied parent MR. Do not author FunctionalRequirements.

SOURCE COVERAGE

List the authorized sources inspected. Do not claim discovery completeness until the declared source set has been checked.

CANDIDATE DISCOVERY

Search for source-supported policy, convention, strategy, architecture or placement boundaries, phase or integration boundaries, lifecycle/data relationships, condition+selected-response structures, and technical choices that may expose a more abstract governed meaning. Discovery is not promotion.

FOR EVERY CANDIDATE**A. SOURCE-SUPPORTED MEANING**

Separate ESTABLISHED / NOT ESTABLISHED / AMBIGUITY / SOURCE GAP.

B. MR STABILITY

Could this meaning change while the parent MR remains the same macro responsibility? PASS | FAIL | HOLD

C. NEUTRALIZATION

Remove concrete product/model/endpoint/protocol/file names, parameter values, thresholds and configuration bindings. Does meaningful project strategy, boundary, policy, convention or placement remain? PASS | FAIL | N/A | HOLD
Surviving neutralization creates a candidate; it does not prove Decision status.

D. SEMANTIC LEVEL

Compare against MR identity, FR/operational obligation, realization, interface, configuration, parameter/binding, data/model fact, verification evidence, observed result and source gap. PASS | FAIL | HOLD

E. INDEPENDENT GOVERNANCE

What original-source evidence establishes that the surviving meaning is owned and governed by the project at this semantic level rather than merely repeated as current realization? PASS | FAIL | HOLD

F. REMOVAL / REPLACEMENT SIGNIFICANCE

If the meaning were removed or replaced while the MR stayed stable, would the authorized basis support that a governed project commitment changed rather than only its realization? PASS | FAIL | HOLD

DECOMPOSITION / SEMANTIC OWNER

Identify separable meanings, but do not create a separate Decision branch only because a meaning has a different conceptual owner. Branch autonomy must later survive the post-FR downstream-utility regression.

PROVISIONAL CLASSIFICATION

KEEP | MERGE | SPLIT | LOWER_LEVEL | REWORK | BOUNDARY_REVIEW |
HOLD / SOURCE_INSUFFICIENT

[FAMILY REGRESSION]

PAIRWISE NON-OVERLAP: PASS | REWORK

UNOWNED GOVERNED MEANING: NONE | ...

BOUNDARY FINDINGS: ...

MR-PROSE LEAKAGE: NONE | ...

FAMILY ACTIONS: ...

POST-FR DOWNSTREAM UTILITY: NOT_TESTABLE_YET | PASS | REWORK

[POST-FR DOWNSTREAM UTILITY TEST]

For each autonomous Decision branch ask:

"If this branch is removed as an autonomous unit and its governed meaning is transferred to the sibling branch that owns the behaviors using it, is any distinction required for implementation, verification, independent change, or analysis lost?"

YES -> autonomous branch may remain.

NO -> reopen MERGE / REWORK. Do not invent an FR to save the branch.

[CLOSURE]

SOURCE INSPECTION: COMPLETE | INCOMPLETE

CANDIDATE DISCOVERY: EXHAUSTED FOR CURRENT SOURCE SET | OPEN

DECISION FAMILY: PROVISIONAL | RESOLVED | HOLD

LOWER-LEVEL SEMANTICS: OPEN | RESOLVED

FR AUTHORIZING: NOT STARTED | STARTED | COMPLETE

[FINAL DECISIONS]

DEC-<id> - <title>

Parent MR: ...

Context: ...

Decision: ...

Consequences: ...

Stop rule

Do not author FunctionalRequirements during Decision discovery. Do not complete source gaps. Do not preserve a Decision branch merely because its statement is true or conceptually distinct. Final branch autonomy requires the post-FR Downstream Utility Test and human semantic review.

5 Step 3 - FunctionalRequirement

5.1 Che cos'è un FunctionalRequirement

Un FunctionalRequirement (FR) descrive un comportamento operativo che il progetto richiede affinché una Decision possa essere realmente rispettata.

La Decision stabilisce una scelta, una regola o un vincolo progettuale. Il FunctionalRequirement chiarisce che cosa il sistema deve fare, produrre o garantire sotto quella scelta.

Per questo un FR non dovrebbe limitarsi a riscrivere la Decision con parole diverse. Allo stesso tempo, non dovrebbe essere reso così astratto da perdere il comportamento che il progetto ha realmente documentato.

Domanda fondamentale

Domanda di review

Quale comportamento operativo deve essere vero perché questa Decision sia soddisfatta?
Quale parte di questo comportamento è effettivamente sostenuta dalle fonti autorizzate?

In questa sezione ricorrono alcuni termini DDTA:

- **owner del significato:** l'elemento documentale che ha la responsabilità di governare quel significato;
- **authority:** la fonte o il livello di governance che ci autorizza a trattare un'informazione come significato del progetto;
- **binding:** un collegamento concreto fra significati che devono rimanere associati, per esempio fra un risultato e la richiesta o il caso a cui appartiene;
- **candidate:** un elemento ancora in valutazione, non ancora accettato;
- **review:** il controllo umano con cui decidiamo se un candidato è corretto, va modificato oppure deve rimanere aperto.

L'obiettivo non è sostituire questi termini, ma fare in modo che il lettore sappia sempre che cosa significano quando vengono usati.

5.2 Dove finisce la Decision e dove inizia il FunctionalRequirement

La distinzione può essere letta in modo semplice:

- la **Decision** risponde a: *quale scelta, regola o vincolo progettuale stiamo adottando?*
- il **FunctionalRequirement** risponde a: *quale comportamento deve essere garantito perché quella scelta sia rispettata?*

Se il testo di un candidato FR può essere sostituito integralmente con quello della Decision senza perdere alcuna informazione, probabilmente stiamo descrivendo due volte lo stesso significato.

In quel caso non dobbiamo forzare la presenza di un FR: dobbiamo riesaminare il confine fra i due livelli.

Se invece le fonti descrivono un comportamento operativo che resta distinguibile dalla scelta espressa nella Decision, quel comportamento può avere una propria identità come FunctionalRequirement.

Domanda di controllo

Domanda di review

Se mantengo invariata la Decision ma elimino questo FR, quale comportamento richiesto dal progetto smette di essere governato?

Se la risposta è "nessuno", il candidato FR deve essere riesaminato.

5.3 Ogni FR ha una sola Decision parent

Ogni FunctionalRequirement appartiene semanticamente a una sola Decision.

Il MacroRequirement di riferimento si ricava risalendo dalla Decision. Non costituisce quindi un secondo parent del FR.

Durante la ricerca delle Decision possono però emergere significati reali che, dopo la review, non vengono accettati come Decision autonome. In questo caso possiamo usare alcune classificazioni provvisorie.

Riclassificato a un livello più basso (LOWER_LEVEL)

Il significato è sostenuto dalle fonti, ma non possiede sufficiente autonomia per essere una Decision. L'informazione non viene eliminata: deve essere conservata e collocata nel livello o presso l'owner più appropriato.

Lasciato in sospeso (HOLD)

Il significato non viene né accettato né scartato. Mancano ancora informazioni, non è chiaro il confine con altri elementi oppure non sappiamo ancora chi debba governarlo. Il candidato resta quindi aperto fino a una review successiva.

LOWER_LEVEL e HOLD **non sono tipi di Decision**. Sono esiti della review di un candidato.

Di conseguenza, un candidato classificato LOWER_LEVEL o HOLD non può diventare parent di un FR finché non viene stabilita una vera Decision che possieda quel comportamento.

5.4 Come individuare gli FR partendo dalle fonti

Per ogni Decision già accettata conviene procedere senza cercare subito di "scrivere requisiti".

Prima ricostruiamo il comportamento che le fonti effettivamente governano.

1. Rileggi la Decision e le fonti autorizzate che la riguardano.
2. Individua le affermazioni minime che descrivono comportamenti, risultati, condizioni o relazioni necessarie.
3. Chiediti quali di queste informazioni descrivono **che cosa deve fare il sistema** e quali descrivono invece **come il sistema è realizzato oggi**.
4. Formula uno o più candidati FR soltanto dopo aver chiarito questa distinzione.
5. Se manca un'informazione, registrala come domanda aperta invece di completarla con una supposizione plausibile.
6. Solo dopo valuta se i candidati devono essere mantenuti, uniti, separati, riclassificati o riesaminati insieme.

Un dettaglio tecnico presente in una tabella, in un diagramma, nel codice o in una configurazione è comunque informazione del progetto. **Non deve essere perso.**

La sua presenza, però, non ci dice automaticamente dove debba essere governato.

Prima di inserirlo nelle clausole normative di un FR dobbiamo capire:

- che cosa significa;
- a quale comportamento è collegato;
- chi ne possiede il significato;
- quale fonte ci autorizza a trattarlo come obbligazione, realizzazione, configurazione o altro elemento documentale.

Questa distinzione serve a collocare correttamente l'informazione, non a ridurla.

5.5 Scheda minima per analizzare un candidato FR

Durante la review può essere utile compilare una scheda molto semplice:

ID del candidato:
 Decision parent:
 Evidenza originale:
 Comportamento operativo candidato:
 Che cosa non sarebbe più governato se lo eliminassi:
 Dettagli di realizzazione/configurazione/binding collegati:
 Possibile altro owner del significato:
 Informazioni mancanti o ambigue:
 Esito provvisorio:

L'esito provvisorio può essere, per esempio:

- mantenere il candidato (KEEP);
- unirlo a un altro (MERGE);
- dividerlo (SPLIT);
- riclassificarlo a un livello più basso (LOWER_LEVEL);
- rielaborarlo (REWORK);
- lasciarlo temporaneamente in sospenso (HOLD).

I codici rendono ripetibile la review ma non sostituiscono la motivazione.

Non inventare un FR per salvare una Decision

La gerarchia non va riempita artificialmente. Se non emerge un comportamento operativo distinto, non creare un FR solo per dare un figlio alla Decision. Al contrario, **riapri la Decision-family review**. Una Decision senza FR propri può restare autonoma solo se il Downstream Utility Test mostra una distinzione necessaria per implementazione, verifica, modifica indipendente o analisi. In assenza di tale utilità, valutare MERGE o REWORK con il branch che possiede il comportamento. Le lacune di fonte restano esplicite e non vanno colmate per supposizione.

5.6 Un FR rappresenta una sola obbligazione coerente

La regola di base è:

```
1 Requirement != 1 frase
1 Requirement = 1 obbligazione normativa coerente
```

Un FR può contenere più clausole, condizioni o un output composto senza essere automaticamente diviso. La domanda non è “quante frasi ci sono?”, ma:

Domanda di review

queste parti descrivono insieme un unico comportamento richiesto oppure rappresentano obbligazioni che il progetto può governare separatamente?

Il numero di punti elenco, campi JSON, tecnologie citate, test o righe di una tabella non decide lo split.

5.7 Quando separare un FR

Valuta la creazione di due FR distinti quando una parte del candidato possiede una vita realmente indipendente dalle altre.

Per esempio, chiediti se quella parte può:

- essere introdotta o rimossa senza cambiare l'altra;
- essere modificata senza cambiare l'identità dell'altra;
- fallire mentre l'altra continua a essere soddisfatta;
- essere verificata e governata separatamente;
- avere un owner differente;
- avere un proprio ciclo di cambiamento o una propria autorità di review.

Nessuno di questi segnali, preso isolatamente, impone automaticamente uno split. Servono a capire se esistono davvero due obbligazioni autonome.

Domanda split/merge

Domanda di review

Se separo questa parte, ottengo un'obbligazione che il progetto governa autonomamente oppure sto soltanto frammentando la descrizione di un unico comportamento?

5.8 Essere verificabile non significa essere un requisito separato

Un FR deve descrivere un comportamento che sia possibile osservare e verificare.

Questo non significa però che ogni parte testabile debba diventare un Requirement distinto.

Due condizioni possono avere test diversi ma continuare ad appartenere alla stessa obbligazione.

Esempio semplice

Supponiamo che un FR richieda di produrre una descrizione strutturata composta da cinque campi.

Possiamo verificare separatamente la presenza e il contenuto dei cinque campi, ma questo non significa che il progetto governi cinque obbligazioni autonome.

Lo split avrebbe senso soltanto se uno o più di quei campi possedessero davvero significato, ownership o ciclo di cambiamento indipendenti.

5.9 Evitare la duplicazione tra Decision e FR

Una Decision non dovrebbe contenere così tanto comportamento operativo da lasciare al FunctionalRequirement soltanto una parafrasi.

Allo stesso modo, non dobbiamo rendere il FR deliberatamente vago solo per farlo apparire diverso dal parent.

Il confine deve emergere dal significato:

- la Decision governa la scelta;
- l'FR governa il comportamento necessario sotto quella scelta.

Domanda di controllo

Domanda di review

Se elimino questo candidato FR lasciando invariata la Decision, quale comportamento richiesto dal progetto non è più vincolato?

Se non riusciamo a identificare alcun comportamento, dobbiamo riesaminare la necessità o la formulazione del candidato.

5.10 Controllare che gli FR coprano la Decision

Dopo aver revisionato i singoli FR bisogna osservare anche la famiglia nel suo insieme.

Immaginiamo che tutti gli FR figli della Decision siano soddisfatti.

Domanda di review

Esiste ancora un comportamento governato necessario alla Decision che nessuno di questi FR copre?

Se sì, possono esserci diverse spiegazioni:

- manca un FR;
- uno degli FR è formulato in modo troppo astratto;
- il comportamento è stato attribuito all'owner sbagliato;
- la documentazione non contiene l'informazione necessaria.

Questo controllo serve a localizzare il problema. Non autorizza a inventare il significato mancante.

5.11 Controllare l'utilità downstream dei branch sibling

I controlli locali restano necessari ma non sufficienti. Una Decision può esprimere una scelta reale e il suo FR può essere formalmente corretto; tuttavia il branch può risultare inutile come **unità autonoma** se, per diventare implementabile, verificabile o analizzabile, deve essere ricongiunto integralmente a un branch sibling.

Branch-level Downstream Utility Test

Dopo aver assemblato gli FR, confrontare il branch completo con i sibling dello stesso MR e chiedere:

Se elimino questo branch come unità autonoma e trasferisco il suo significato nel branch che possiede i comportamenti che lo utilizzano, perdo una distinzione necessaria per implementazione, verifica, modifica indipendente o analisi?

YES: il branch resta autonomo. **NO:** riaprire MERGE/REWORK. Il test non chiede se la Decision sia “vera”, ma se la sua autonomia documentale produca valore downstream.

DermaTriage - worked example completo, parte 1/2: branch semanticamente valido ma sospetto

I due branch hanno lo stesso parent: **MR-01 - Valutazione di triage del caso dermatologico.**

Branch A - DEC-MR01-02, Scala di priorità P1-P4.

Decision: “DermaTriage usa la scala P1-P4 per rappresentare la priorità operativa di triage.”

Consequence: la priorità operativa è uno dei livelli P1, P2, P3 o P4; le regole per assegnare il livello sono definite a parte.

FR-MR01-02-01 - Dominio della priorità P1-P4.

Functional obligation: ogni priorità operativa è rappresentata mediante P1, P2, P3 o P4.

Normative clause: quando viene prodotta una priorità, il valore MUST appartenere a {P1, P2, P3, P4}.

La Decision non è falsa: governa una scelta reale di dominio. Il problema emerge downstream: il suo FR ripete quel dominio ma non introduce un comportamento autonomo che possa essere implementato, verificato o analizzato senza il branch che determina concretamente la priorità.

DermaTriage - worked example completo, parte 2/2: branch operativo e controfattuale

Branch B - DEC-MR01-04, separazione tra urgenza analitica e priorit  operativa.

Decision: DermaTriage mantiene distinta la valutazione analitica dell'urgenza dalla priorit  operativa di triage; quest'ultima   determinata successivamente e rappresentata mediante la scala P1-P4.

Consequence: la priorit  non   l'esito analitico stesso:   un risultato operativo distinto, appartenente al dominio P1-P4, derivato dall'urgenza e dalle ulteriori informazioni governate applicabili.

FR-MR01-04-01 - Mapping urgenza/confidence verso P1-P4.

Condizione	Priorit� richiesta
HIGH + confidence > 0.85	MUST produrre P1
HIGH, negli altri casi applicabili	MUST produrre P2
MEDIUM	MUST produrre P3
LOW	MUST produrre P4

Downstream Utility Test sul Branch A. Rimuovendo DEC-MR01-02 e FR-MR01-02-01 come ramo autonomo e trasferendo la scelta P1-P4 nel contratto di DEC-MR01-04, non si perde alcuna distinzione necessaria per codice, test, modifica indipendente o analisi. Il dominio resta governato e il comportamento che lo rende effettivo resta nell'FR di mapping. Il Branch A era quindi **semanticamente valido ma strutturalmente superfluo**.

Consolidazione pratica risultante

La Decision mantenuta governa un solo contratto operativo coerente: trasformare un esito analitico in una **priorit  operativa distinta** appartenente al dominio P1-P4. Il FunctionalRequirement definisce la funzione concreta che associa urgenza/confidence ai quattro livelli. DEC-MR01-02 non viene eliminata perch  "sbagliata", ma perch  la sua autonomia non aggiunge utilit  downstream; FR-MR01-02-01 non giustifica il branch perch  ripete soltanto il codominio gi  necessario al mapping.

Conservazione prima di accettare merge o rimozione

Prima della consolidazione verificare che restino rappresentati commitment, comportamenti, dominio, condizioni e binding materiali e che i gap restino visibili. Qui restano aperte, per esempio, l'applicabilit  del mapping al percorso symptom-only e la semantica degli eventuali tempi/SLA associati a P1-P4.

5.12 Una lacuna nella documentazione non è un requisito

Quando per comprendere il comportamento completo serve un'informazione che le fonti non stabiliscono, quella mancanza deve essere resa esplicita.

Non dobbiamo creare un FR per nascondere la lacuna o per rendere l'albero apparentemente completo.

Esempio DermaTriage

Durante la review di MR-01 abbiamo trovato tre significati documentati:

- il triage può continuare anche senza immagine usando le informazioni sintomatologiche disponibili;
- la priorità operativa appartiene al dominio P1-P4;
- è documentata una regola che usa urgenza e confidence per selezionare un valore P1-P4 in un determinato percorso.

Le fonti analizzate non stabiliscono però che l'urgenza prodotta dal percorso symptom-only debba attraversare necessariamente quella stessa regola.

La risposta corretta non è inventare un ottavo FR.

La risposta corretta è registrare la domanda aperta:

Domanda di review

Come viene determinata la priorità P1-P4 quando l'urgenza proviene dal percorso symptom-only?

Finché le fonti non rispondono, il significato resta una **lacuna documentale aperta**.

5.13 Obbligazione funzionale e dettagli della realizzazione

Questa distinzione è delicata perché non vogliamo né confondere il requisito con l'implementazione né perdere informazioni utili sul sistema reale.

Per ogni informazione chiediamoci quindi due cose diverse:

1. **Qual è il comportamento che il progetto obbliga a garantire?**
2. **Come viene realizzato oggi quel comportamento?**

Le due risposte possono cambiare in momenti diversi.

Un FR descrive anzitutto il comportamento governato. La documentazione deve però conservare anche ciò che sappiamo della realizzazione corrente: modelli, componenti, tecnologie, parametri, endpoint, formati, configurazioni e binding.

Separare non significa cancellare.

Se, per esempio, un comportamento viene oggi realizzato da un particolare modello ML, il nome del modello non deve scomparire dalla documentazione solo perché non definisce necessariamente l'identità del FunctionalRequirement.

Dobbiamo invece capire quale ruolo ha quel dettaglio.

- Se le fonti stabiliscono che il progetto **impone proprio quel dettaglio** come parte dell'obbligazione, esso può entrare nella semantica normativa dell'FR.
- Se il dettaglio descrive **come l'obbligazione è realizzata oggi**, deve restare documentato e collegato all'FR, ma non diventa automaticamente parte dell'identità del requisito.
- Se le fonti contengono il dettaglio ma non è ancora chiaro quale sia il suo ruolo normativo, lo manteniamo visibile e lo segnaliamo come punto da chiarire.

La domanda quindi non è:

Domanda di review

"Questo dettaglio è troppo tecnico e va eliminato?"

ma:

Domanda di review

"Che cosa rappresenta questo dettaglio e dove deve essere governato affinché non vada perso?"

Alcuni vincoli o proprietà possono richiedere un proprio requisito specializzato quando rafforzano le condizioni di soddisfazione del FR senza sostituirne l'obbligazione ordinaria. Il SecurityRequirement resta la specializzazione security già governata; lo Step 4 di questa guida introduce e verifica anche il ConfigurationRequirement per le proprietà configurative che superano i relativi gate.

Questa sezione non promuove automaticamente un dettaglio tecnico a requisito specializzato. Fissa il principio di preservazione e rinvia allo Step 4 la classificazione normativa delle configurazioni.

5.14 Nessuna informazione supportata dalle fonti deve andare persa

L'astrazione e la neutralizzazione servono a capire **dove appartiene un'informazione**, non a cancellarla.

Quando incontriamo un fatto supportato dalle fonti possiamo trovarci in quattro situazioni.

5.14.1 1. Il fatto appartiene direttamente all'obbligazione funzionale

In questo caso può essere espresso nelle clausole normative dell'FR.

5.14.2 2. Il fatto appartiene a un altro owner o a un'altra fase del ciclo di vita

L'informazione deve essere conservata presso quell'owner e collegata al comportamento a cui si riferisce.

Per esempio, un parametro di training può riguardare l'evoluzione del modello e non il comportamento runtime governato dallo specifico FR.

5.14.3 3. Il fatto è documentato, ma non sappiamo ancora se sia normativo

Non lo promuoviamo automaticamente a requisito, ma non lo eliminiamo. Rimane visibile come informazione supportata dalle fonti in attesa di una classificazione migliore.

5.14.4 4. L'informazione necessaria non è documentata

Non possiamo inventarla. Registriamo invece una lacuna esplicita.

In pratica, durante la review non dobbiamo chiederci soltanto:

Domanda di review

"Questa informazione appartiene all'FR?"

Dobbiamo anche chiederci:

Domanda di review

"Se non appartiene all'FR, dove la documentiamo e come manteniamo il collegamento con il comportamento a cui si riferisce?"

Quando qualcosa cambia

Se cambia soltanto la realizzazione, la configurazione o un binding, aggiorniamo la parte di documentazione che descrive quel dettaglio e la relativa tracciabilità.

Se cambia invece il comportamento obbligatorio governato dal progetto, riesaminiamo anche il FunctionalRequirement.

Se una tecnologia concreta è stata resa esplicitamente parte dell'obbligazione dalle fonti, allora anche la sua sostituzione può richiedere una modifica dell'FR.

5.15 Contratto L1 minimo

Il metamodel già governato resta la base. Questa revisione non introduce nuovi campi nel FunctionalRequirement.

```
Requirement [abstract]
    normativeClause : NormativeClause [1..*]

FunctionalRequirement extends Requirement
    parentDecision : Decision [1]
```

Questo significa che ogni FR:

- è un Requirement;
- possiede una o più normativeClause;
- ha esattamente una Decision parent.

Le informazioni aggiuntive utilizzate per rendere la documentazione leggibile non devono diventare nuovi elementi del metamodel senza una decisione metodologica esplicita.

5.16 Come presentare un FR in modo leggibile

La rappresentazione documentale può mostrare in modo leggibile:

- ID;
- stato di lifecycle;
- authority;
- tipo FunctionalRequirement;
- titolo;
- Decision parent;
- MacroRequirement derivato dal parent, quando aiuta la lettura;
- una sintesi dell'obbligazione funzionale;
- una o più normativeClause;
- riferimenti strutturati ed evidenze già governate;
- eventuali SpecializedRequirement realmente stabiliti.

Questi elementi aiutano una persona a capire il requisito, ma non devono creare una seconda semantica parallela rispetto al metamodel.

Informazioni come trigger, input, output, criteri di verifica, realizzazione corrente e domande aperte possono essere molto utili durante l'autoring e la review.

La loro utilità, però, non basta a trasformarle automaticamente in nuovi campi L1.

Devono essere documentate usando le strutture e i meccanismi di tracciabilità appropriati.

5.17 Come scrivere le clausole normative

Le clausole normative devono rendere chiaro che cosa il progetto richiede.

Per gli obblighi usiamo **MUST** e **MUST NOT**.

SHOULD e **MAY** hanno significati diversi:

- **SHOULD** esprime una raccomandazione governata;
- **MAY** esprime un permesso governato.

Non vanno usati per attenuare un obbligo solo perché non siamo sicuri di ciò che la fonte intende.

Se non sappiamo se qualcosa sia obbligatorio, il problema è nell'autorità o nella documentazione e deve rimanere visibile.

La descrizione leggibile della **Functional obligation** aiuta il lettore a comprendere l'FR, ma la fonte normativa primaria del Requirement rimane l'insieme coerente delle sue normativeClause.

5.18 Controllo finale del candidato FR

Prima di accettare un candidato, prova a rispondere con evidenze esplicite a queste domande:

1. Quale parte delle fonti sostiene questo comportamento?
2. Quale unica Decision lo possiede?
3. Sto descrivendo un comportamento operativo oppure sto ripetendo la Decision?
4. Se elimino l'FR, quale comportamento richiesto smette di essere governato?
5. L'obbligazione possiede sufficiente autonomia da giustificare una propria identità?
6. L'eventuale split separa davvero due obbligazioni oppure frammenta artificialmente un unico comportamento?
7. Sto trasformando una tecnologia, un parametro, una rappresentazione o un binding in obbligo normativo senza sufficiente supporto?
8. Ogni valore concreto inserito nelle clausole normative è realmente sostenuto dalle fonti?
9. L'insieme degli FR copre il comportamento governato dalla Decision?
10. Le informazioni mancanti rimangono esplicite oppure le abbiamo completate per supposizione?

Al termine della review il candidato può essere:

- mantenuto (KEEP);
- unito a un altro (MERGE);
- separato (SPLIT);
- riclassificato a un livello più basso (LOWER_LEVEL);
- rielaborato (REWORK);
- lasciato in sospeso (HOLD).

Il codice sintetizza l'esito. La motivazione e l'evidenza spiegano perché quell'esito è corretto.

Nessuna decisione deve dipendere dal raggiungimento di un numero atteso di FR.

Controllo obbligatorio - percorso dati e contratti informativi

Dopo aver identificato i FunctionalRequirement di una Decision, enumerare ogni informazione prodotta, consumata, resa disponibile o persistita. Per ogni coppia producer/consumer verificare se esiste un transfer semanticamente rilevante. Ogni transfer candidato deve poter rendere identificabili source, destination e content; quando il segmento deve essere analizzato, qualificato, realizzato o protetto autonomamente, deve poter ricevere identità'. Per ogni contenuto trasferito deve essere verificata la sufficienza del relativo information/data contract. Informazioni necessarie ma non stabilite dalla fonte devono restare esplicitamente NOT SPECIFIED / diagnostic gap e non essere ricostruite per plausibilità'.

Questo controllo non introduce nuovi campi L1 e non impone un FunctionalRequirement per ogni scambio: transfer, contratto informativo, binding e realization devono ancora essere classificati secondo source support, semantic owner e livello corretto.

5.19 Controllare la famiglia completa degli FR

Anche quando ogni FR sembra corretto individualmente, la famiglia può avere problemi. Dopo la review dei singoli requisiti verifica quindi che:

- ogni FR abbia una sola Decision parent;
- due FR non governino inutilmente lo stesso comportamento;
- un unico comportamento non sia stato frammentato senza motivo;
- nessun significato operativo governato sia rimasto senza owner;
- i dettagli di realizzazione, configurazione e binding siano ancora presenti e tracciabili;
- le lacune ancora aperte siano visibili;
- eventuali modifiche alle Decision non abbiano reso obsoleti i relativi FR;
- ogni branch Decision ancora autonomo abbia superato il **Downstream Utility Test**.

Feedback obbligatorio verso la Decision family

Se una Decision non possiede FR propri, oppure i suoi FR non introducono alcun comportamento downstream distinto, non inventare requisiti e non chiudere automaticamente il ramo. Riportare il branch alla Decision-family review e verificare se la sua autonomia serve davvero a implementazione, verifica, modifica indipendente o analisi. In caso contrario valutare MERGE o REWORK preservando il significato nel branch operativo corretto.

Il numero degli FR è un risultato, non un obiettivo. Una famiglia può contenerne più o meno rispetto a una versione precedente se le fonti o la comprensione del significato lo giustificano.

5.20 Confrontare la derivazione con un'analisi indipendente

Una guida è più utile se persone diverse, partendo dalle stesse fonti, ricostruiscono significati compatibili. Possiamo quindi chiedere a un altro ricercatore di derivare gli FR indipendentemente e confrontare i risultati. Lo scopo non è stabilire chi “ha vinto”, ma individuare:

- punti ambigui nelle fonti;
- passaggi poco chiari nella guida;
- criteri di split/merge interpretati diversamente;
- informazioni tecniche collocate in livelli differenti;
- eventuali esempi della guida che hanno orientato troppo la risposta.

L'analisi esterna e l'output di un LLM restano **evidenze di interpretazione**, non authority di progetto. La maggioranza non decide il significato: per riconciliare le divergenze si torna alle fonti, all'owner del significato e ai controlli previsti dalla guida.

5.21 Come classificare le differenze tra due analisi

Quando due derivazioni non coincidono, conviene descrivere **su che cosa** stanno divergendo prima di tentare di risolvere il problema.

La tassonomia usata nel lavoro sperimentale comprende:

- **split o merge** (SPLIT_MERGE): gli analisti non concordano su quante obbligazioni autonome esistano;
- **authority della fonte** (SOURCE_AUTHORITY): non concordano su quale fonte autorizzi un significato;
- **Decision o FR** (FR_VS_DECISION): lo stesso significato viene collocato a livelli diversi;
- **FR o realizzazione** (FR_VS_REALIZATION): un dettaglio viene interpretato da uno come requisito e dall'altro come realizzazione;
- **FR o binding** (FR_VS_BINDING): diverge la classificazione di un collegamento concreto;
- **FR o parametro** (FR_VS_PARAMETER): diverge il ruolo di un valore o parametro;
- **FR o specializzazione** (FR_VS_SPECIALIZATION): non è chiaro se il significato appartenga alla funzione ordinaria o a una proprietà specializzata;
- **ownership** (OWNERSHIP): non c'è accordo su chi debba governare il significato;
- **significato mancante** (MISSING_SEMANTICS): una delle analisi evidenzia informazioni necessarie ma non documentate;
- **ambiguità della guida** (GUIDE_AMBIGUITY): la stessa regola metodologica permette letture differenti;
- **influenza degli esempi** (GUIDE_EXAMPLE_ANCHORING): un esempio della guida può avere suggerito la soluzione;
- **differenza solo rappresentativa** (REPRESENTATIONAL_ONLY): cambia la forma ma non il significato;
- **altro** (OTHER): differenza non coperta dalle categorie precedenti.

Queste etichette non risolvono la divergenza. Servono a renderla osservabile e confrontabile.

5.22 Preparare una replica realmente indipendente

Se vogliamo capire quanto la guida sia ripetibile, il ricercatore indipendente non deve conoscere in anticipo la soluzione che vogliamo verificare.

Nel pacchetto di lavoro vanno quindi evitati, per quanto possibile:

- vecchi ID degli FR del progetto target;
- numero atteso di requisiti;
- esempi già risolti sullo stesso progetto;
- commenti che rivelano quale split o merge sia stato accettato;
- gap formulati in modo tale da suggerire già la classificazione finale.

Il ricercatore deve poter arrivare alla propria derivazione usando le fonti e le regole metodologiche disponibili.

Una sequenza utile è:

```
derivazione indipendente
|
confronto dei significati
|
classificazione delle differenze
|
riconciliazione umana tornando alle fonti
|
minima correzione necessaria della guida
|
nuovo controllo sullo stesso ramo
|
nuova replica indipendente senza suggerimenti sulla risposta
```

In questo modo le differenze tra le analisi diventano uno strumento per capire **dove la metodologia o la documentazione sono ancora ambigue**.

5.23 Uso di un LLM come supporto all'autoring

Un LLM può aiutare a cercare candidati, organizzare evidenze e applicare in modo ripetibile le domande della guida.

Non può però creare significato di progetto.

Il modello deve quindi seguire gli stessi principi dell'autore umano:

- usare soltanto le fonti autorizzate per stabilire il significato;
- non trasformare automaticamente i dettagli tecnici in requisiti;
- non eliminare dettagli documentati solo perché sono classificati come realizzazione o configurazione;
- non usare vecchi FR come prova che una nuova derivazione sia corretta;
- non colmare informazioni mancanti con conoscenza di dominio o con una soluzione plausibile;
- rendere visibili dubbi e lacune invece di nasconderli.

Il prompt canonico per i FunctionalRequirement rimane uno strumento operativo separato dalla spiegazione metodologica.

La guida deve spiegare **come ragionare**; il prompt deve aiutare il modello ad applicare quel ragionamento in modo controllato.

6 Step 4 - SpecializedRequirement e ConfigurationRequirement

6.1 SpecializedRequirement: quando serve un requisito aggiuntivo

Uno `SpecializedRequirement` governa una obbligazione concern-specific aggiuntiva che deve valere insieme a un singolo `FunctionalRequirement`. Non sostituisce la funzione ordinaria del parent e non esiste per classificare un dettaglio tecnico: deve aggiungere una condizione normativa autonoma di soddisfazione.

```
SpecializedRequirement [abstract]
  extends Requirement
  parentFunctionalRequirement : FunctionalRequirement [1]
```

$\text{Effective}(F) = F \text{ AND } S1 \text{ AND } \dots \text{ AND } S_n$

Gate di strengthening e removal

Per ogni candidato chiedere:

- esiste esattamente un FR parent riconoscibile?
- rimuovendo il candidato, l'obbligazione funzionale ordinaria del FR resta riconoscibile?
- il candidato rafforza le condizioni di soddisfazione invece di riscrivere il comportamento ordinario?
- l'obbligazione concern-specific puo' essere revisionata o assessed indipendentemente dal parent senza cambiarne l'identita' funzionale?
- la fonte governa davvero questo significato come obbligazione e non soltanto come realizzazione corrente?

Se una risposta decisiva e' negativa, non creare uno `SpecializedRequirement` per ragioni di ordine documentale.

Requirement ownership e usage scope sono assi distinti

Il Requirement che governa la necessita', disponibilita' o configurazione di una risorsa, capability o artifact mantiene il proprio owner anche quando l'elemento risultante viene usato trasversalmente da piu' componenti o branch. L'uso condiviso non moltiplica il parent del Requirement e non trasferisce automaticamente ownership ai consumer.

Una libreria software, un dataset, un servizio condiviso o una risorsa di infrastruttura possono quindi essere giustificati e governati da un Requirement specifico e successivamente utilizzati da molte parti del sistema. Le relazioni di uso/consumo restano un problema distinto dalla gerarchia documentale del Requirement.

6.2 ConfigurationRequirement

Definizione

Un ConfigurationRequirement e' uno SpecializedRequirement che rafforza un singolo FunctionalRequirement imponendo proprieta' o condizioni configurative governate sotto le quali l'obbligazione del parent e' considerata soddisfatta. La configurazione puo' riguardare risorse hardware, software, runtime, binding, deployment o altri parametri solo quando la loro natura normativa e' source-supported.

La prima integrazione metodologica usa il delta L1 minimo:

```
ConfigurationRequirement
    extends SpecializedRequirement
```

Inherited:

```
parentFunctionalRequirement : FunctionalRequirement [1]
normativeClause             : NormativeClause [1..*]
```

Non viene introdotto in questa fase un campo L1 obbligatorio per il profilo. Un eventuale ConfigurationProfile e' una rappresentazione strutturata e tracciabile del contratto configurativo, disciplinata separatamente.

Gate ConfigurationRequirement

ACCEPT solo se sono sostenuti tutti i punti necessari:

1. **Parent survival:** rimuovendo il ConfigurationRequirement, il comportamento/capability governato dal FR resta riconoscibile.
2. **Normative configuration:** la fonte stabilisce che almeno una proprieta' configurativa deve o dovrebbe valere; una baseline concreta da sola non basta.
3. **Independent change/assessment:** la configurazione puo' cambiare o essere verificata senza ridefinire l'identita' funzionale del parent.
4. **Realization boundary:** versioni, package, file, locator o valori correnti non diventano automaticamente obblighi normativi.
5. **Ownership boundary:** il requisito di configurazione non acquisisce automaticamente il significato funzionale degli elementi cui i binding fanno riferimento.

Esiti di review consigliati:
 ACCEPT_CONFIGURATION_REQUIREMENT
 PROFILE_ONLY
 REALIZATION_ONLY
 HOLD / SOURCE_INSUFFICIENT

Un profilo software che elenca Python, librerie e versioni puo' quindi essere obbligatorio da preservare come evidenza tecnica senza dimostrare che ogni versione costituisca un ConfigurationRequirement.

6.3 ConfigurationProfile come contratto leggibile e machine-readable

Un **ConfigurationProfile** raccoglie in forma compatta fatti configurativi che appartengono allo stesso contesto operativo. Può essere presentato come tabella, file strutturato o altra serializzazione machine-readable e può ricevere un identificatore documentale stabile. Questo non lo rende automaticamente un nuovo tipo L1 né un BAREferent.

Classificare ogni entry, non uniformare il profilo

L'appartenenza allo stesso profilo non rende omogenea la semantica delle entry. Ogni voce deve conservare la propria classe e il proprio owner. Il profilo può contenere contemporaneamente obblighi, raccomandazioni, current realization, binding/references e gap, purché la distinzione resti esplicita.

Una forma L2 utile è:

```
ConfigurationProfile <id>
  scope / concern: <...>
  parent ConfigurationRequirement: <id | None>

  entries:
    - key: <property/binding/component>
      value: <source-supported value or ->
      semanticClass:
        REQUIRED | RECOMMENDED | CURRENT_REALIZATION |
        REFERENCE_BINDING | GAP | CONFLICTING
      semanticOwner: <document owner | unresolved>
      sourceEvidence: <reference>
```

Se il ConfigurationRequirement demanda i valori al profilo, la prosa normativa deve rendere identificabile quale sottoinsieme è vincolante. Le entry CURRENT_REALIZATION, REFERENCE_BINDING, GAP o CONFLICTING non diventano vincoli solo perché compaiono nello stesso artifact.

Punto di incontro documentazione-codice

Quando utile, la stessa rappresentazione strutturata può essere versionata, letta da tooling, usata per generare o validare configurazioni e richiamata dalla documentazione. L'automazione non cambia l'authority: il significato normativo continua a provenire dai Requirement e dalle fonti autorizzate; la serializzazione ne rende il contratto ispezionabile e verificabile.

Se due fonti assegnano valori incompatibili alla stessa entry, il profilo registra CONFLICTING; non normalizza automaticamente il dato scegliendo il valore più plausibile.

6.4 Software composition inventory, BOM/SBOM e rilevanza security

Versioni, librerie, package e componenti software concreti possono essere **security-sensitive anche quando non sono Requirement normativi**. La loro sostituzione puo' cambiare superficie d'attacco, esposizione a vulnerabilita', dipendenze transitive e condizioni di supporto. Per questo la classificazione come current realization **non autorizza a perdere l'informazione**.

Regola di preservazione security-sensitive

Quando le fonti autorizzate identificano componenti software, package o versioni, la documentazione deve mantenerli tracciabili in un inventario di composizione software o BOM/SBOM equivalente, usando soltanto i dati effettivamente supportati. Un campo non noto resta non specificato; non viene completato con conoscenza esterna durante la ricostruzione source-first.

Stato corrente e configurazione futura sono distinti

Lo stato corrente di una realizzazione puo' essere obbligatorio da documentare e analizzare senza essere obbligatorio come configurazione futura del sistema. Quando un ConfigurationProfile/BOM fotografa la baseline effettivamente analizzata, la documentazione puo' esplicitare `profileStatus = CURRENT_IMPLEMENTATION`: questo rende la baseline tracciabile per security e change-impact analysis, ma non trasforma le entry in Requirement normativi.

Quando build, installazione o deployment usano un artifact machine-readable (per esempio un dependency manifest o lock file), la documentazione dovrebbe riferirsi a quell'artifact operativo. Se il profilo concretizza una configurazione logica governata, l'handoff alla BA applica l'identity test a configurazione astratta e realizzazione concreta: quando entrambe hanno identita' BA e la fonte supporta la materializzazione, il legame usa l'operatore BA gia' governato realize; non si introduce un operatore separato `currentImplementationOf`. Se uno dei gate non passa, resta soltanto traceability documentale. In ogni caso il legame non rende la realizzazione corrente un vincolo futuro. Se l'artifact non e' nel source set, contenuto e allineamento restano da verificare.

L'inventario puo' essere un ConfigurationProfile software o un artifact collegato. La distinzione da preservare e':

```
package/version presente nella baseline
  -> CURRENT_REALIZATION / software inventory

package/version imposto dalla fonte
  -> candidate normative configuration

obbligo di mantenere, validare o proteggere una BOM/SBOM
  -> candidate Requirement concern-specific
      da classificare secondo il concern source-supported
```

La rilevanza per la threat analysis non crea retroattivamente un commitment di progetto. Un package puo' essere importante per cercare CVE, attacchi di supply chain o impatti di sostituzione e rimanere comunque una realization concreta. Viceversa, se il progetto governa esplicitamente la presenza, versione, provenienza, integrita' o aggiornamento di un componente, quel significato puo' richiedere un ConfigurationRequirement, un SecurityRequirement o altro Requirement appropriato secondo la fonte.

Handoff verso security e change impact

L'inventario software deve permettere, quando la fonte lo consente, di risalire almeno all'identita' del componente e alla versione/variante documentata. Ulteriori attributi come source, hash, licenza o dipendenze vengono conservati solo quando disponibili o richiesti. La guida di authoring preserva l'informazione; la successiva analisi di sicurezza decide come usarla senza diventare authority del progetto.

6.5 Regression DermaTriage e procedura di authoring

I test sul branch C6 mostrano tre esiti differenti e impediscono una promozione meccanica:

Caso	Esito	Lettura metodologica
FR-C6-01-01 hardware	ConfigurationRequirement: PASS candidate	RAM/storage hanno valori minimum/recommended esplicitamente governati; il FR resta riconoscibile come disponibilit� delle risorse anche rimuovendo il profilo.
FR-C6-01-02 software stack	PROFILE/INVENTORY: PASS; ConfigurationRequirement: HOLD	package e versioni sono preservati e security-sensitive, ma la source non stabilisce che ogni versione sia un commitment normativo stabile.
FR-C6-01-04 binding ambiente	ConfigurationRequirement: PASS candidate con profilo misto	alcuni binding sono richiesti, mentre altri valori rinviano a owner funzionali differenti; l'appartenenza al profilo non trasferisce ownership.

Sequenza deterministica

1. accettare prima il FunctionalRequirement e verificare che possieda una funzione/capability autonoma;
2. cercare proprieta' concern-specific che rafforzano la sua soddisfazione;
3. applicare i gate SpecializedRequirement e ConfigurationRequirement;
4. costruire, se utile, un ConfigurationProfile e classificare ogni entry;
5. preservare comunque software inventory/BOM e altre realization security-sensitive;
6. solo dopo consegnare la documentazione alla Base Analysis.

6.6 Handoff alla Base Analysis

Il nuovo tipo documentale non introduce automaticamente un nuovo semantic operator BA. La BA continua a estrarre identita' e proposition secondo il significato: per esempio constrain, hasPart, realize o altri operatori gia' governati. Un ConfigurationProfile riceve identita' BA solo se supera l'identity test della BA; una riga del profilo non diventa una BAPosition per il solo fatto di essere presente.

Le correzioni alla BA devono quindi essere riestratte dalla documentazione aggiornata, non introdotte nel documento per ottenere una topologia grafica desiderata.

Indice di integrità' delle pagine

Regola di verifica

Ogni pagina possiede un MD5 del proprio contenuto canonico. Il calcolo usa il contenuto LaTeX compreso tra i marker PAGE-CONTENT; il valore MD5 stampato sulla pagina è escluso dal payload della pagina stessa. Nell'indice, gli hash delle altre pagine sono risolti prima del calcolo, mentre l'hash della pagina indice viene sostituito dal token canonico <SELF> per evitare una dipendenza circolare.

Pag.	Contenuto	MD5 contenuto
1	Frontespizio della nuova guida di authoring	7289db2d0692a47b1d1013b12077997f
2	Project Problem Framing - definizione e regole di authoring	015c757c3dc4118c78c6d2960996936e
3	Project Problem Framing - procedura umana, gate ed esempio DermaTriage	22e57dfc4b4f586235f8aed3c522a7e2
4	Project Problem Framing - LLM Execution Profile e prompt canonico	3bdac4b5fec97501c23d7968bf1619d5
5	Project Problem Framing - regola LLM e classificazione dei campi	b69ae801bbd9fab0dd9722c5416054c4
6	Project Problem Framing - formato esatto dell'output LLM	dd6326ef067e8650d8c2fb33662caf1b
7	Project Problem Framing - gate LLM e nota sui controlli quantitativi	8828e008581efc3d40954e198b8db8c0c
8	Uso della metodologia - authoring nativo e reconstruction	39dd4da3676b9d5a04cebbd9a196c627
9	MacroRequirement - definizione e rapporto con framing e Decision	ddcc227fc8f2ef02b24235ea1a2979c66
10	MacroRequirement - procedura umana per individuare i candidati	fee9637e13dfb1e7c117390773a8e3ae
11	MacroRequirement - test di separazione, split e merge	97e6e0faea39d999309befffb6aca06bd
12	MacroRequirement - campi documentali e regole di compilazione	70297884e3aacfc8f40b330ff7183ca4
13	MacroRequirement - template, split merge e STOP AT MR	9a97d1490def76a4e8d0cd67a8307a34
14	MacroRequirement - sufficienza semantica e diagnosi dei problemi	b008d2293cbe637ae1268cc43c3b2351
15	MacroRequirement - profilo LLM e prompt canonico parte 1	af9687992a8bac51cb56ea985aeb2ebc
16	MacroRequirement - prompt canonico LLM parte 2	fab634987bb3e565df9c1d1f7ad066d0
17	MacroRequirement - prompt LLM campi e schema di analisi	bc1bb076c245425bda75ebd8a2cb5c6
18	MacroRequirement - formato esatto output LLM e gate finale	a70c8156b1d0d0dd6c5640b2a8564
19	Decision - definizione confine e discovery	62e667a1c7db0a35813bdb36a8115714
20	Decision - worksheet candidate e source support	d240cc1ea1cb104187dcd2ce3d89b046
21	Decision - MR stability neutralization semantic level	da7ba071ce540507ac258349820dd96d
22	Decision - independent governance e removal replacement	a70410d72d2daf48666b8f2addac2407
23	Decision - decomposition, semantic owner e utilita' downstream	2aed4a907b4eccc2027ecc165ad9dacc
24	Decision - esempio completo e MR prose leakage	df4e6b117b67e0101047ac7b62483340
25	Decision - family regression, downstream utility e closure	8a63264d93dc6926960764a7b017eb86
26	Decision - LLM execution profile prompt canonico parte 1	e03bf85cd279f18b7e7b73a2028473
27	Decision - prompt LLM candidate gates	1f804382f39d12572469d7fb1269df71
28	Decision - prompt LLM decomposition, downstream regression e output	651725f978e9d26d4bf4a4c05c31f933
29	FunctionalRequirement - definizione, termini e domanda fondamentale	052bb1e5051941bc7f21945436cffa4e
30	FunctionalRequirement - confine Decision-FR, parent unico e classificazioni di review	f5310549d08de40525cf85e2df1b4e16
31	FunctionalRequirement - individuazione source-first dei comportamenti	8921ecd8cfa9bf31d7449e9c7da8ebbd
32	FunctionalRequirement - scheda candidate, branch utility e obbligazione coerente	2565613776f5cf4b81ef70950950637d
33	FunctionalRequirement - split e verificabilità senza frammentazione	2541b4df42daadc5fb2402472d4e8c10
34	FunctionalRequirement - non duplicazione e copertura della Decision	255ec7ee9d4053db3e78fe65baaf4216
35	FunctionalRequirement - downstream utility regression e DermaTriage branch A	ddda2e275d829cad56c4dad9a70d4ae
36	FunctionalRequirement - DermaTriage downstream utility e consolidazione pratica	f15a3bb4200cf549e5828495b75c4384
37	FunctionalRequirement - lacune documentali ed esempio DermaTriage	e4a9d2dd8df5e956a76592038b883979
38	FunctionalRequirement - obbligazione funzionale e dettagli della realizzazione	8b94f41bede4a3f9786ab2efb9d33ed5
39	FunctionalRequirement - information preservation e change impact	76fa920eae92f2bb829a2743a84b30a8
40	FunctionalRequirement - contratto L1 e presentazione L2	e7136268d32bf6086596d5f030a947ad
41	FunctionalRequirement - scrittura delle clausole normative	9bc0e63e85d2d026b925a81ded38de52
42	FunctionalRequirement - controllo finale del candidato	cc250cd2a2deb3069dd6892e943bd284
43	FunctionalRequirement - controllo famiglia, downstream utility e analisi indipendente	9c03ac3e778c27333ef2d51de02f6eb2
44	FunctionalRequirement - classificazione delle differenze tra analisi	fbcb7bc514bb10a6d2c69b4728ff01ba
45	FunctionalRequirement - replica indipendente e supporto LLM	e7298e5d8dc77d5ea99741209730c0c
46	SpecializedRequirement - strengthening, parent unico e ownership distinta dallo usage scope	5f5b0c26e4dacb02b8f71d3721b126bb
47	ConfigurationRequirement - definizione, contratto minimo e gate di accettazione	3e4344fc346cd8db52a9d36deb9c1e3b
48	ConfigurationProfile - rappresentazione tipizzata, machine-readable e confine tra normative e realization	e20b6314acab5f7db4f66b3254fe6590
49	Software BOM e version traceability - preservazione security-sensitive senza promozione normativa automatica	26686d9f09a34bef1278db0f985789e6
50	ConfigurationRequirement - regression DermaTriage, authoring sequence e handoff alla Base Analysis	89ce247f911f19826656691138e0c416
51	Indice di integrità' delle pagine	9527d376948e281c7fc1ed809a6849b9

A ogni passo, una modifica intenzionale cambia l'MD5 della pagina interessata. Una variazione inattesa dell'MD5 di una pagina già approvata impone la revisione della modifica prima di proseguire.